Systems Research & Development (R&D) Report

Project: IT Resource & Operations Platform (IROP)
Author: Engineering Intern
Date: November 2026
Document Type: Architectural Research & Comparative System Analysis

---

Executive Summary

Modern IT and software engineering companies require streamlined operational tracking across three foundational administrative areas:

1. **Leave Management (Human Resources)**
2. **Asset Management (IT Hardware & Software Licensing)**
3. **Inventory Management (Consumables & Office Supplies)**

Rather than adopting three fragmented, third-party subscription tools that lead to data silos, this research explores the architectural design, feature sets, and optimal technology stacks for building each of these three systems into a cohesive, high-performance operational portal.

---

1. Leave Management System (LMS)

1.1 Overview & Core Purpose

The Leave Management System automates employee time-off requests, balances, and multi-tier managerial approval workflows. It eliminates the manual errors, email threads, and spreadsheet discrepancies common in ad-hoc leave tracking.

1.2 Key Features

- **Role-Based Leave Workflows**: Employees submit leave requests; direct reporting managers review team requests; Administrators maintain company-wide oversight.
- **Dynamic Weekday Calculator**: Automatically computes working days (Mondays through Fridays), excluding weekends (Saturdays and Sundays).
- **Strict Overlap Prevention**: Validates incoming dates and rejects overlapping pending or approved requests for the same employee.
- **Atomic Quota Deductions**: Deducts quotas only upon managerial approval using atomic database operations to prevent race conditions.
- **Real-Time Quota Balances**: Real-time tracking of Total, Used, and Remaining allowances across multiple leave categories (Casual, Sick, Annual, and Work-From-Home).
- **Cancellation Lifecycle**: Employees can cancel pending requests without penalty before review.

1.3 Proposed Technology Stack

- **Frontend**: React 18 with TypeScript & Tailwind CSS
- **Backend API**: Node.js & Express 5
- **Database**: MongoDB with Mongoose ODM
- **State Management & Data Fetching**: TanStack Query (React Query)
- **Validation**: Zod (runtime request/response schema validation)

1.4 Reasons for Choosing This Stack

1. **Atomic `$inc` Operators for Race-Condition-Free Deductions**: When multiple users or automated background services interact with leave balances, concurrent approvals can cause double-deductions. MongoDBΓÇÖs native `$inc` operator guarantees atomic updates at the document level without complex SQL table locking.
2. **Nested Document Storage for Quotas**: Each employee has multiple quota buckets (`casual` , `sick` , `annual` , `wfh`). In MongoDB, these are embedded directly inside the `User` document, allowing instant retrieval of the employee profile and all balances in a single, high-speed database query.
3. **Reactive UI for Date Calculations**: React provides immediate client-side date arithmetic. As the employee picks start and end dates on the calendar, the component calculates and displays the working day count in real time before submission.

---

2. Asset Management System (AMS)

2.1 Overview & Core Purpose

The Asset Management System manages the complete lifecycle of corporate IT hardware (laptops, monitors, workstations) and digital assets (software licenses, SaaS seats). It establishes clear custody chains and prevents asset loss or license non-compliance.

2.2 Key Features

- **Dual Physical & Digital Asset Support**: Tracks both physical equipment (with serial numbers and specs) and digital software licenses (with license keys and seat counts).
- **Status Lifecycle Control**: Strict transitions between `Available` , `In Use` , and `In Repair` .
- **Assignment Safeguards**: Assets marked `In Repair` are locked from assignment until restored to `Available` . Assigning an asset automatically sets its status to `In Use` ; unassigning restores it to `Available` .
- **Debounced Search & Filtering**: Real-time search across asset names, serial numbers, and license keys with status dropdown filtering.
- **Role-Restricted Visibility**: Employees see only the assets assigned to them; Managers and Admins access full company inventory.

2.3 Proposed Technology Stack

- **Frontend**: React 18 with TypeScript & Lucide Icons
- **Backend API**: Node.js & Express 5
- **Database**: MongoDB with Sparse Unique Indexes
- **Client Cache**: TanStack Query with optimistic mutations

2.4 Reasons for Choosing This Stack

1. **Polymorphic Data Modeling**: Hardware assets possess physical hardware specs (RAM, CPU, serial number, purchase date), while software licenses possess digital attributes (license keys, seats, expiration dates). MongoDBΓÇÖs flexible BSON document model allows both categories to exist harmoniously in a single `assets` collection without messy SQL foreign keys or null columns.
2. **Sparse Unique Indexes**: Hardware items require unique serial numbers, whereas bulk un-serialized accessories (e.g., HDMI adapters) do not. MongoDBΓÇÖs `sparse: true` unique index enforces uniqueness when a serial number is provided while permitting un-serialized accessories without index collisions.
3. **Debounced Search Performance**: Node.js and Express handle text search queries with sub-millisecond response times, paired with client-side debouncing in React to eliminate unnecessary server load.

---

3. Inventory Management System (IMS)

3.1 Overview & Core Purpose

The Inventory Management System manages recurring office supplies, consumables, and computer peripherals (cables, keyboards, chargers, stationery). It ensures continuous stock availability and prevents operational bottlenecks.

3.2 Key Features

- **Stock & Threshold Tracking**: Tracks item quantities alongside minimum threshold levels.
- **Low-Stock Alert Badges**: Triggers a visual alert badge whenever `quantity <= threshold` (including exact matches).
- **One-Click Quick Controls**: Administrative users can increment or decrement item quantities via one-click `+` and `ΓêÆ` controls.
- **Zero-Floor Enforcement**: Database and API constraints guarantee quantities cannot drop below zero (`quantity >= 0`).
- **Categorical Grouping**: Items organized by category (Peripherals, Cables, Stationery, Power).
- **Role-Based Access Control (RBAC)**: Full CRUD for Administrators; read-only visibility for Team Managers; Employees restricted from warehouse inventory.

3.3 Proposed Technology Stack

- **Frontend**: React 18 with Tailwind CSS
- **Backend API**: Node.js & Express 5
- **Database**: MongoDB with Mongoose Schema Validators

3.4 Reasons for Choosing This Stack

1. **Rapid Counter Modifications**: Stock adjustments (e.g., clicking `+` or `ΓêÆ` when handing out a cable) are frequent and lightweight. Node.js non-blocking I/O combined with MongoDB atomic increment operations provides instantaneous response times.
2. **Schema-Level Min Constraints**: Mongoose schema validation (`min: [0, 'Quantity cannot be negative']`) enforces the zero-floor rule directly at the data model layer, preventing invalid inventory states regardless of client inputs.
3. **Optimistic UI Updates**: Using TanStack Query on the React frontend, the UI reflects quantity changes immediately upon user click while synchronizing with the backend in the background.

---

4. Architectural Synthesis: The Unified MERN Platform Advantage

Building these three systems as unified modules within a single **MERN (MongoDB, Express, React, Node.js + TypeScript)** platform provides distinct engineering advantages over deploying isolated tools:

Dimension	Isolated Tools (SaaS / Multi-Stack)	Unified MERN Platform (IROP)
User Identity	3 separate logins, accounts, and passwords	1 central JWT authentication system with shared role management
Data Relationships	Complex sync pipelines or webhooks	Direct references: Employees link to assigned assets, managers, and leave records
Development Velocity	Multiple languages and frameworks	Single language (TypeScript) across frontend, API, and database models
Deployment Footprint	Multiple servers, distinct databases	Single Docker Compose orchestration or unified cloud deployment
Maintenance Cost	High recurring subscription overhead	Zero licensing fees, fully open and self-hosted

5. Conclusion

The research confirms that combining Leave Management, Asset Management, and Inventory Management into a single full-stack MERN application is both architecturally sound and operationally superior. The unified document model of MongoDB, the asynchronous throughput of Node/Express, and the component reactivity of React deliver an enterprise-grade operational hub for modern IT organizations.